

Busca em profundidade

“Esgotar o filho antes de esgotar o pai”

“A próxima aresta a ser visitada parte sempre do vértice **mais recente** na busca”

Busca em profundidade

Complexidade da busca em profundidade:

$$O(n + m)$$

(onde $n = |V(G)|$ e $m = |E(G)|$)

Busca em profundidade

Se o grafo for desconexo, a busca alcançará apenas os vértices que estão conectados ao vértice-raiz da busca por caminhos!!

Alternativa:

modificar a chamada externa para alcançar todo o grafo

*enquanto existe v em $V(G)$ tal que $PE(v) = 0$ faça
executar $P(v)$*

Busca em profundidade

- Uma **árvore** é um grafo conexo e acíclico (sem ciclos).
- Se o grafo de entrada G é conexo, a árvore de profundidade T é uma **árvore geradora de G** (isto é, uma árvore que alcança todos os vértices de G); neste caso, todos os vértices de G são alcançados pela busca e ficam com uma PE diferente de zero no final da mesma.
- Somente as arestas **azuis** (ligando pai e filho) pertencem à árvore de profundidade T . As arestas **vermelhas** (arestas de retorno) não pertencem a T .

Busca em profundidade

Propriedades das arestas de retorno

- Toda aresta de retorno fecha um **ciclo**.
- Toda aresta de retorno liga um vértice v a um de seus **ancestrais** na árvore de profundidade T .

Busca em profundidade

v	a	b	c	d	e	f	g
$PE(v)$	3	4	2	5	1	10	11
$PS(v)$	8	7	9	6	14	13	12

$t \rightarrow$	1	2	3	4	5	6	7	8	9	10	11	12	13	14
a														
b														
c														
d														
e														
f														
g														

Intervalos de vida dos vértices: v é descendente de w na árvore de profundidade T se e somente se o intervalo de vida de v está contido no intervalo de vida de w .
 Isto é: $PE(v) > PE(w)$ e $PS(v) < PS(w)$ (v “entra depois” e “sai antes” de w).

Busca em profundidade

Aplicação 1: Dado um grafo G , verificar se G é conexo.

Solução: Rodar o laço abaixo.

$c \leftarrow 0$

enquanto existe v em $V(G)$ tal que $PE(v) = 0$ faça

$c \leftarrow c + 1$

executar $P(v)$

fim-enquanto

No final da execução, a variável c é igual ao número de **componentes conexas** de G (subgrafos conexos maximais que compõem G).

Busca em profundidade

Aplicação 2: Dado um grafo G e dois vértices v, w de G , verificar se existe um caminho de v a w em G .

Solução: Basta executar uma única vez o procedimento $P(v)$. No final da execução, se $PE(w) = 0$ então w não foi alcançado pela busca com raiz v , isto é, w está em uma componente conexa diferente da de v , e portanto não existe caminho de v a w em G . Caso contrário, se $PE(w) \neq 0$, então w foi alcançado, e existe um caminho de v a w em G ; neste caso, w será descendente de v na árvore de profundidade T , e para determinar o caminho basta rodar o algoritmo a seguir.

Busca em profundidade

Aplicação 2: Dado um grafo G e dois vértices v, w de G , verificar se existe um caminho de v a w em G .

Solução (continuação):

$x \leftarrow w$

$C \leftarrow x$

enquanto $x \neq v$ faça

$x \leftarrow \text{pai}(x)$

colocar x à esquerda em C

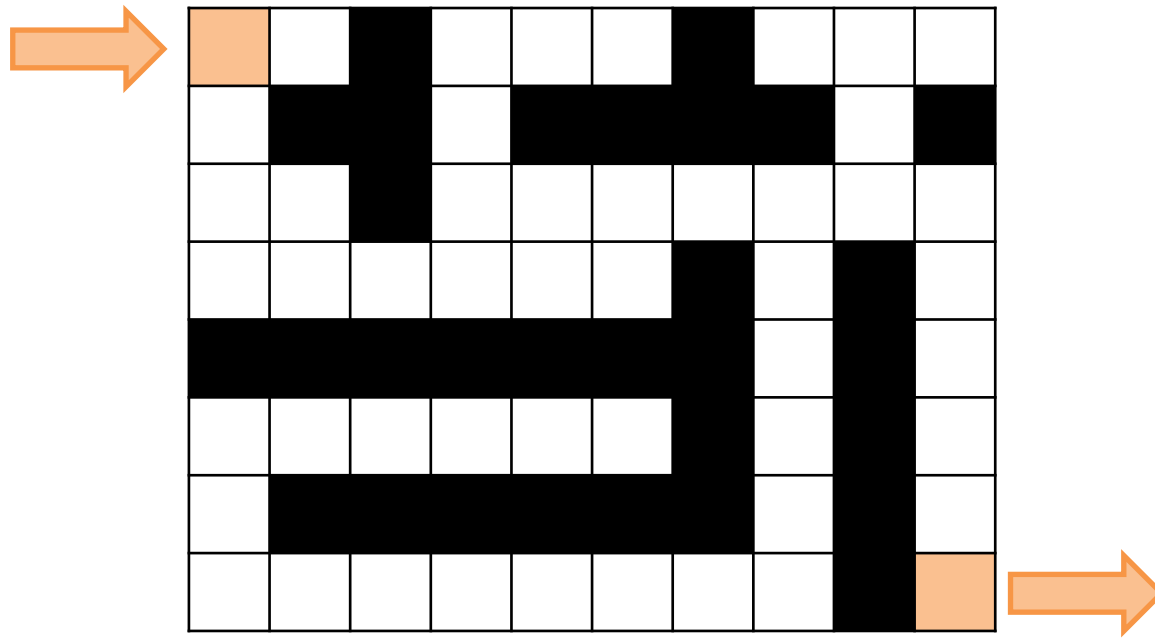
fim-enquanto

imprimir C

Obs: C não é necessariamente o melhor

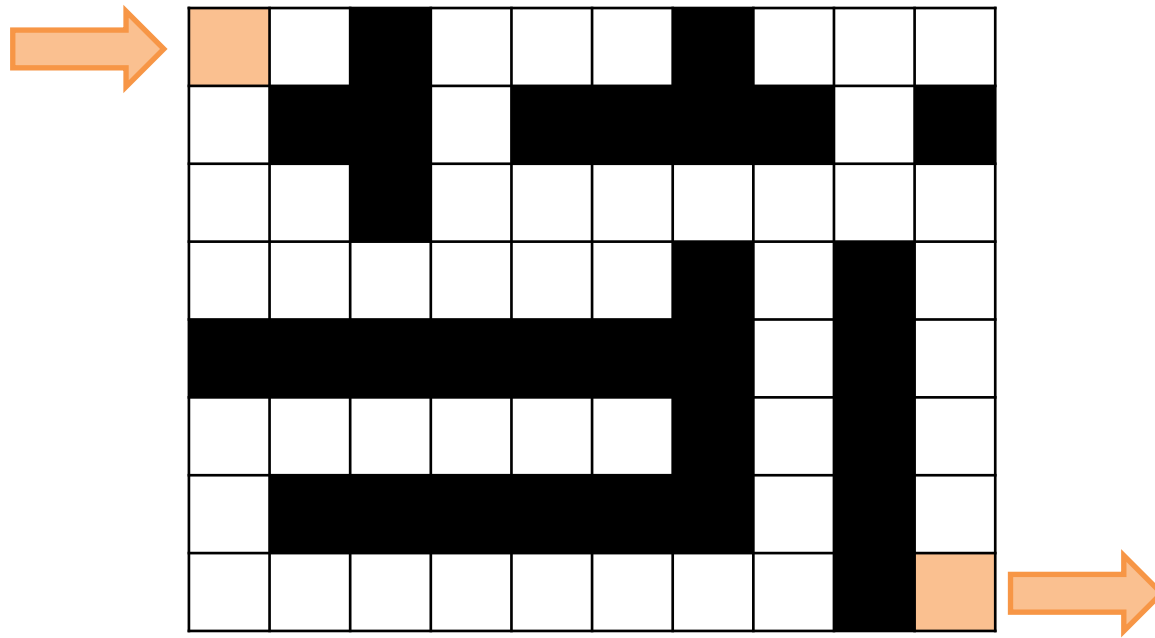
Busca em profundidade

Aplicação 3: Dado um labirinto, determinar um caminho da entrada até a saída (se existir).



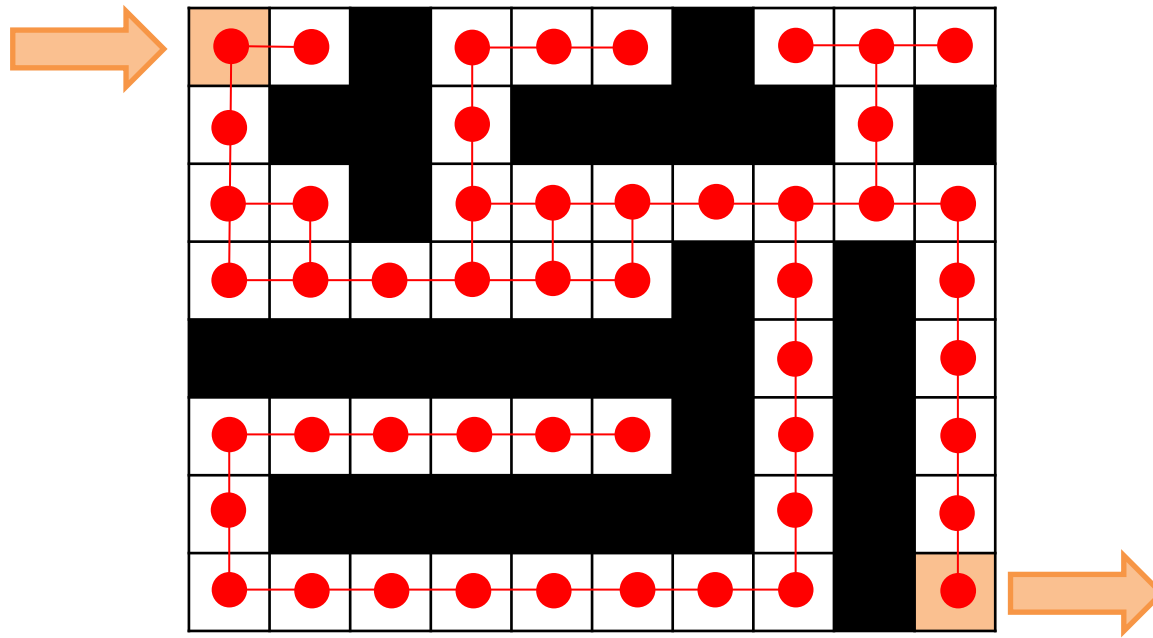
Busca em profundidade

Aplicação 3 - Solução: Montar o grafo correspondente ao labirinto e usar a solução da aplicação anterior.



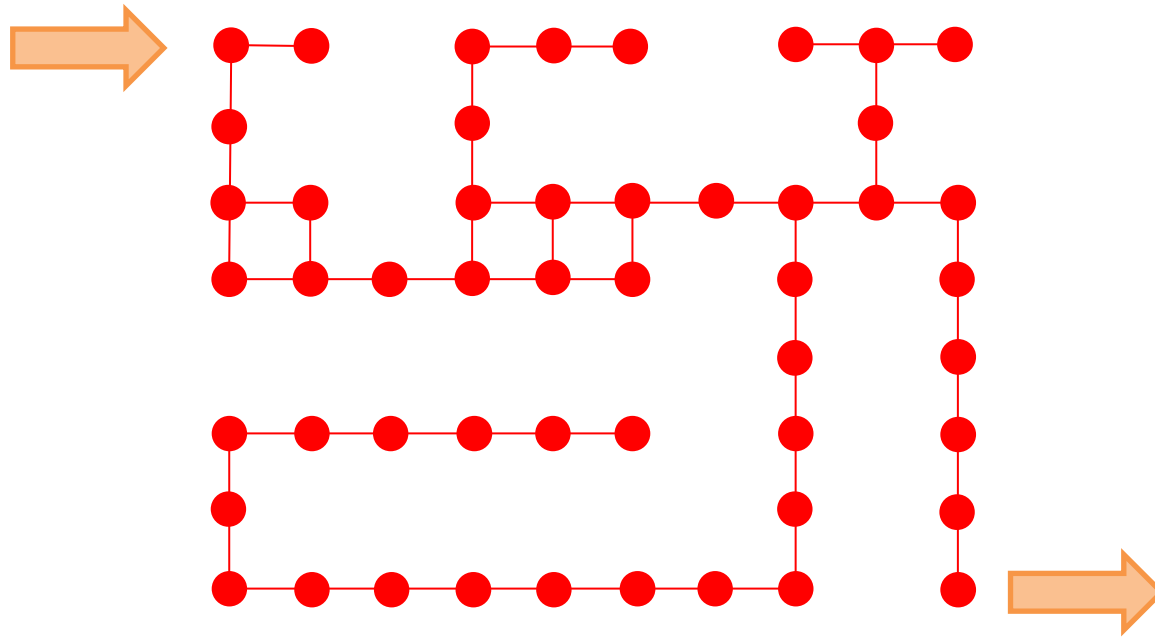
Busca em profundidade

Aplicação 3 - Solução: Montar o grafo correspondente ao labirinto e usar a solução da aplicação anterior.



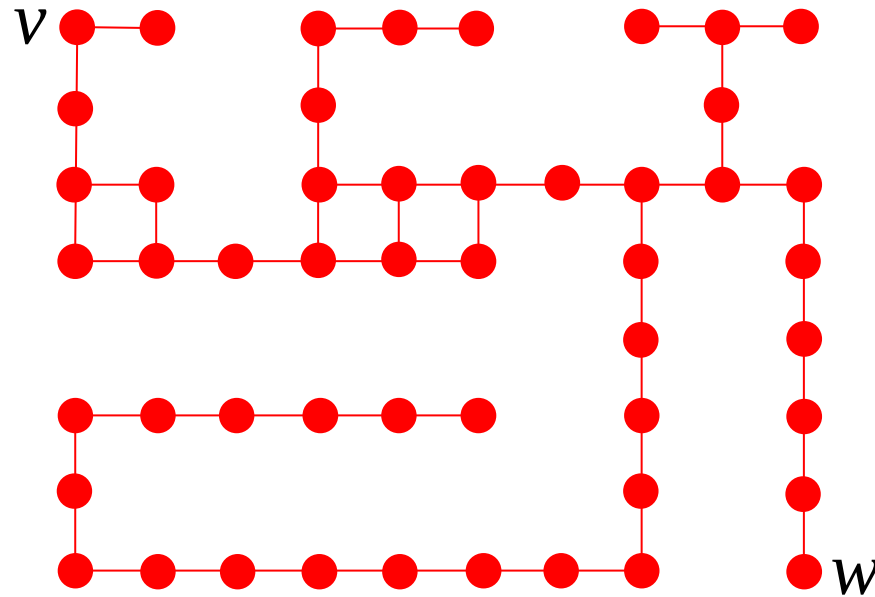
Busca em profundidade

Aplicação 3 - Solução: Montar o grafo correspondente ao labirinto e usar a solução da aplicação anterior.



Busca em profundidade

Aplicação 3 - Solução: Montar o grafo correspondente ao labirinto e usar a solução da aplicação anterior.



Busca em profundidade

Aplicação 4: Dado um grafo G , encontrar um ciclo de G ou concluir que G é acíclico.

Solução: Executar uma busca em profundidade em G .
Teremos dois casos:

- A busca não gerou nenhuma aresta de retorno. Então, G é acíclico, e as arestas **azuis** formam uma **floresta geradora** (um conjunto de árvores, uma para cada componente conexa).
- A busca gerou uma aresta de retorno **vw** . Então, v é descendente de w na árvore, e um ciclo C é formado. Para determinar C , basta rodar o algoritmo a seguir:

Busca em profundidade

Aplicação 4: Dado um grafo G , encontrar um ciclo de G ou concluir que G é acíclico.

Solução (continuação):

- $x \leftarrow v$

$C \leftarrow x$

enquanto $x \neq w$ faça

$x \leftarrow \text{pai}(x)$

colocar x à direita em C

fim-enquanto

colocar v à direita em C (para fechar o ciclo)

imprimir C

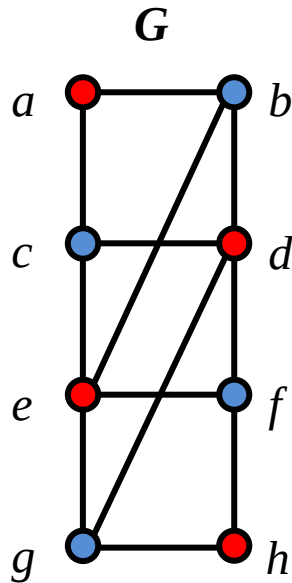
Busca em profundidade

Aplicação 5: Dado um grafo G , decidir se G é 2-colorível.

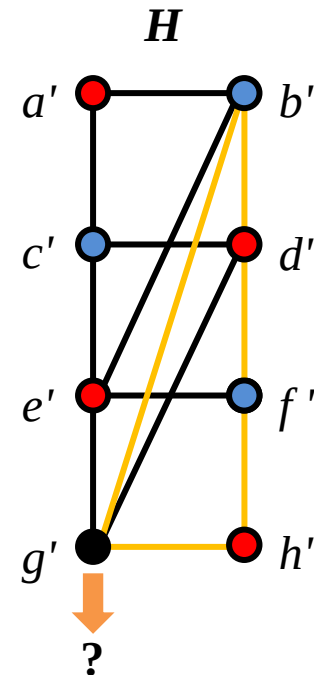
- Um grafo G é **2-colorível** (ou **bipartido**) se podemos colorir os vértices de G com duas cores de modo que vértices vizinhos não recebam a mesma cor.
- **Teorema:** G é 2-colorível se e somente se G não contém um ciclo ímpar (ciclo com número ímpar de arestas).

Busca em profundidade

Aplicação 5: Dado um grafo G , decidir se G é 2-colorível.



2-colorível



não 2-colorível

Busca em profundidade

Aplicação 5: Dado um grafo G , decidir se G é 2-colorível.

Solução: Atribuir cor “0” à raiz da busca e rodar:

procedimento $P(v)$

$t \leftarrow t + 1; PE(v) \leftarrow t$

para todo vértice w em $N(v)$ faça

se $PE(w) = 0$

então visitar vw ;

$pai(w) \leftarrow v; cor(w) \leftarrow 1 - cor(v);$

executar $P(w)$

senão se $PS(w) = 0$ e $w \neq pai(v)$

então se $cor(w) \neq cor(v)$

então visitar vw

senão pare: ciclo ímpar! (imprimir como na Aplic.4)

fim-para

$t \leftarrow t + 1; PS(v) \leftarrow t$